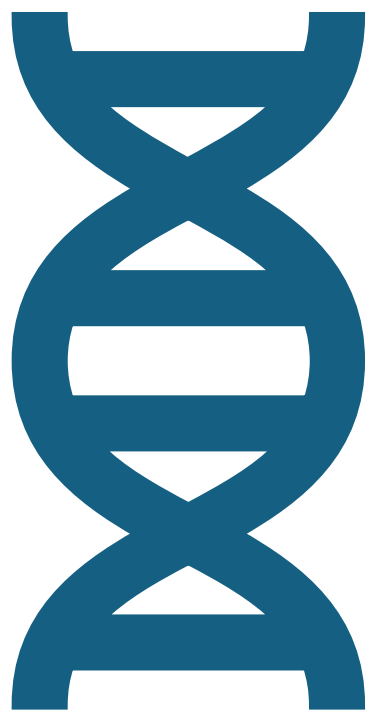




# Генетический алгоритм

Команда 16: Иванов Александр,  
Крапивина Ульяна





**Генетические алгоритмы** — семейство эвристических методов безусловной оптимизации. Они решают задачу нахождения глобального экстремума (минимума или максимума) некой целевой функции  $f(\vec{x})$ ,  $\vec{x} \in \mathbb{R}^n$ .

ГА хорошо подходят для сложных функций, к которым не применимы более простые алгоритмы по той или иной причине, например многомодальность или сложность вычисления градиента.



## Постановка задачи

Что дано?

Пусть задана целевая функция  $f(x)$ , определённая на множестве допустимых решений  $D \subset \mathbb{R}^n$ :

$$f(x) = f(x_1, x_2, \dots, x_n), \quad x \in D.$$

Требуется найти глобальный экстремум (минимум или максимум):

$$f(x^*) = \min_{x \in D} f(x) \quad \text{или} \quad f(x^*) = \max_{x \in D} f(x).$$

## В чём проблема?

Классические методы требуют:

- непрерывности и дифференцируемости функции;
- хорошего начального приближения;
- унимодальности (одного экстремума).

Однако на практике в машинном обучении мы часто сталкиваемся с функциями, которые:

- разрывны (например, точность классификации);
- многоэкстремальны (много локальных оптимумов);
- не дифференцируемы (например, число ошибок);
- заданы «чёрным ящиком» (нет аналитического выражения).

## Что требуется?

Разработать эвристический метод поиска, который:

1. Не требует производных и аналитического вида функции.
2. Работает с разрывными и многоэкстремальными функциями.
3. Находит достаточно хорошее решение (не обязательно глобальный оптимум) за приемлемое время.
4. Может быть распараллелен.

Именно эту задачу решает генетический алгоритм.



Области применения

## В машинном обучении

- **Настройка гиперпараметров:** подбор `max_depth`, `learning_rate`, `C`, `gamma` для моделей, где градиентные методы неприменимы.
- **Отбор признаков (Feature Selection):** поиск оптимального подмножества признаков из тысяч возможных.
- **Обучение нейронных сетей:** эволюционная стратегия для весов сети (альтернатива backpropagation в задачах с недифференцируемой функцией потерь).
- **Генетическое программирование:** синтез математических выражений, решающих деревьев, программ.
- **Кластеризация:** поиск оптимального числа кластеров и начальных центроидов (гибрид с K-means).
- **Составление ансамблей:** выбор лучших моделей и их весов.

## За пределами ML

- Задача коммивояжёра: поиск кратчайшего маршрута.
- Составление расписаний: распределение задач по исполнителям.
- Оптимизация запросов в базах данных: планирование порядка соединений.
- Искусственная жизнь: моделирование экосистем и эволюции.
- Генеративное искусство: эволюционный дизайн.



Основная идея

|                    |                                         |
|--------------------|-----------------------------------------|
| Биология           | Генетический алгоритм                   |
| Особы              | Одно решение задачи $x$                 |
| Хромосома          | Закодированное решение (строка, вектор) |
| Ген                | Один параметр решения $x_i$             |
| Популяция          | Набор из $N$ решений                    |
| Приспособленность  | Значение целевой функции $f(x)$         |
| Естественный отбор | Селекция лучших решений                 |
| Скрещивание        | Обмен частями хромосом                  |
| Мутация            | Случайное изменение гена                |
| Эволюция           | Итеративное улучшение популяции         |

**«Выживает сильнейший»** — особи с лучшими значениями функции приспособленности имеют больше шансов передать свои гены потомкам. Со временем популяция эволюционирует в сторону всё лучших решений.



Математический  
аппарат

## 1. Кодирование решений (генотип)

Каждое решение  $x = (x_1, \dots, x_n)$  кодируется в хромосому  $s = (s_1, \dots, s_L)$ , где  $L$  — длина хромосомы.

Примеры кодирования:

- Бинарное:  $x_i \in \{0, 1\}$ ,  $s$  — битовая строка.
- Вещественное:  $x_i \in \mathbb{R}$ ,  $s$  — вектор вещественных чисел.
- Перестановка: для задачи коммивояжёра  $s$  — порядок городов.

Преобразование (декодирование):

$$x = e^{-1}(s), \quad s = e(x),$$

где  $e$  — функция кодирования,  $e^{-1}$  — декодирования.

## 2. Функция приспособленности

Функция приспособленности  $\Phi(s)$  — мера качества особи. Связана с целевой функцией  $f(x)$ :

$$\Phi(s) = \varphi(f(e^{-1}(s))),$$

где  $\varphi : \mathbb{R} \rightarrow \mathbb{R}^+$  — монотонно возрастающая функция.

- Для задачи максимизации:  $\Phi(s) = f(x(s))$ .
- Для задачи минимизации:  $\Phi(s) = 1 + f(x(s))$  (или другая монотонно убывающая замена).

### 3. Вероятность селекции (пропорциональный отбор)

Вероятность выбора особи  $s_i$  пропорциональна её приспособленности:

$$P(s_i) = \frac{\Phi(s_i)}{\sum_{j=1}^N \Phi(s_j)},$$

где  $N$  — размер популяции.

Формула пропорциональной селекции с константой  $C$  (для задач минимизации):

$$P(X^i) = \frac{-fitness(X^i) + C}{r \cdot C - \sum_{j=1}^{\lfloor r \rfloor} fitness(X^j)}, \quad r - \text{размер популяции}, C : P(X^i) \geq 0 \forall i$$

**Элитарная селекция:** хотя бы одна копия наилучшего решения всегда проходит отбор. Это гарантирует, что лучшая найденная особь не будет потеряна из-за случайных операторов.

**Ранговая селекция:** решения ранжируются в соответствии с функцией цели или функцией приспособленности. Вероятность выбора зависит от ранга, а не от абсолютного значения приспособленности, что снижает давление отбора при наличии «суперособей».

**Турнирная селекция:** создаются группы случайных решений размером  $t \geq 2$ . Из каждой группы выбирается наиболее приспособленное решение. Проста в реализации, легко распараллеливается, устойчива к масштабу функции приспособленности.

#### 4. Оператор кроссинговера (одноточечный)

Для двух родителей  $s^1 = (s_1^1, \dots, s_L^1)$  и  $s^2 = (s_1^2, \dots, s_L^2)$  выбирается точка разрыва  $\chi \in \{1, \dots, L - 1\}$ :

$$s'^1 = (s_1^1, \dots, s_\chi^1, s_{\chi+1}^2, \dots, s_L^2),$$

$$s'^2 = (s_1^2, \dots, s_\chi^2, s_{\chi+1}^1, \dots, s_L^1).$$

Другие виды кроссинговера:

- **Двухточечный кроссинговер:** хромосомы родителей делятся на три части двумя случайными точками и рекомбинируются. Хромосома потомства имеет две части от одного родителя и одну часть от другого.
- **Многоточечный кроссинговер:** аналогично, с  $n$  точек и  $n - 1$  участков.

## 5. Оператор мутации

Каждый ген с вероятностью  $p_m$  инвертируется:

$$s'_i = \begin{cases} 1 - s_i, & \text{с вероятностью } p_m, \\ s_i, & \text{иначе.} \end{cases}$$

## 6. Теорема о схемах (Schema Theorem)

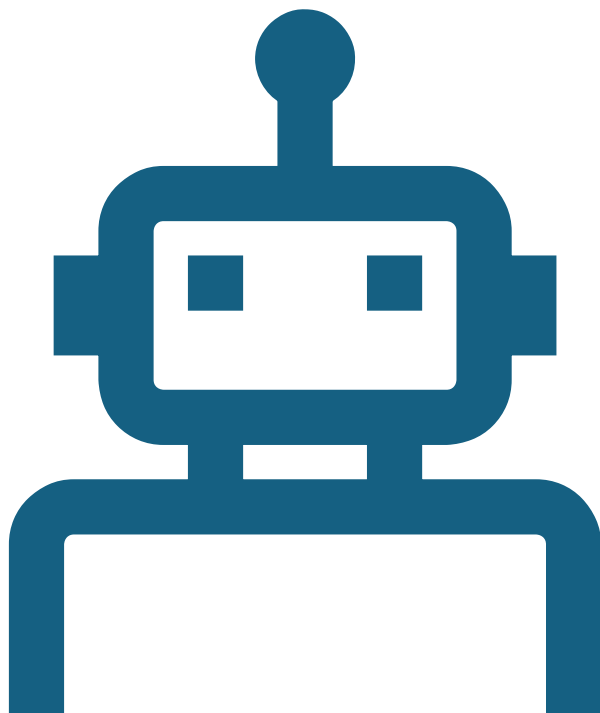
Схема  $H$  — это шаблон, например,  $H = (1, *, 0, *, 1)$ , где  $*$  — любой символ.

- Порядок схемы  $o(H)$  — число фиксированных позиций.
- Длина схемы  $\delta(H)$  — расстояние между крайними фиксированными позициями.

Теорема Холланда:

$$m(H, t+1) \geq m(H, t) \cdot \frac{\Phi(H, t)}{\Phi(t)} \cdot \left[ 1 - p_c \frac{\delta(H)}{L-1} - o(H)p_m \right],$$

где  $m(H, t)$  — число представителей схемы  $H$  в поколении  $t$ ,  $\Phi(t)$  — средняя приспособленность популяции.



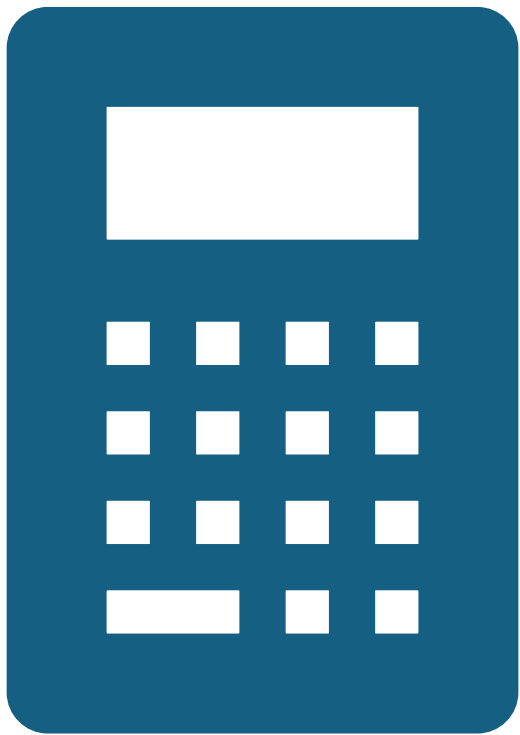
Алгоритм

## Общая схема

1. **Инициализация.** Создать начальную популяцию  $P_0 = \{s_1, \dots, s_N\}$  случайным образом.
2. **Оценка.** Вычислить приспособленность каждой особи:  $\Phi(s_i)$  для всех  $i = 1, \dots, N$ .
3. **Проверка условия остановки.** Если выполнено (достигнут максимум поколений  $T$ , найдено решение достаточного качества, истекло время), перейти к шагу 8.
4. **Селекция.** Выбрать  $N$  родительских особей с вероятностями, пропорциональными их приспособленности.
5. **Скрещивание.** С вероятностью  $p_c$  для каждой пары родителей применить оператор кроссинговера.
6. **Мутация.** Для каждого потомка с вероятностью  $p_m$  применить оператор мутации к каждому гену.
7. **Формирование нового поколения.** Заменить старую популяцию потомками (или использовать элитарную стратегию).
8. **Конец.** Вернуть лучшую найденную особь.

## Рекомендуемые параметры

| Параметр                        | Диапазон   | Комментарий                                          |
|---------------------------------|------------|------------------------------------------------------|
| Размер популяции $N$            | 20–100     | Меньше — быстрее, но риск преждевременной сходимости |
| Вероятность кроссинговера $p_c$ | 0.6–0.95   | Высокая — больше новых комбинаций                    |
| Вероятность мутации $p_m$       | 0.001–0.01 | Низкая — сохраняет лучшее, но не даёт застрять       |
| Число поколений $T$             | 50–1000    | Зависит от сложности задачи                          |



Численный пример

## 1. Постановка задачи и начальная популяция

### Задача

Найти глобальный максимум функции

$$f(x) = 2x^2 + 1$$

на отрезке целых чисел  $x \in \{0, 1, 2, \dots, 31\}$ .

### Кодирование

Каждое число  $x$  представим в виде 5-битной двоичной строки:

$$x = 19 \Rightarrow s = 10011_2$$

Начальная популяция (случайная,  $N = 6$ )

| № | Хромосома $s$ | $x = e^{-1}(s)$ | $f(x) = 2x^2 + 1$ | $\Phi(s)$ |
|---|---------------|-----------------|-------------------|-----------|
| 1 | 10011         | 19              | 723               | 723       |
| 2 | 00011         | 3               | 19                | 19        |
| 3 | 00111         | 7               | 99                | 99        |
| 4 | 10101         | 21              | 883               | 883       |
| 5 | 01000         | 8               | 129               | 129       |
| 6 | 11101         | 29              | 1683              | 1683      |

Суммарная приспособленность:

$$\sum_{i=1}^6 \Phi(s^i) = 723 + 19 + 99 + 883 + 129 + 1683 = 3536.$$

Средняя приспособленность:

$$\Phi = \frac{3536}{6} \approx 589.3.$$

## 2. Селекция (метод рулетки)

Вероятности выбора:

$$P(s^i) = \frac{\Phi(s^i)}{\sum_{j=1}^6 \Phi(s^j)}.$$

| № | $\Phi(s^i)$ | $P(s^i)$ | Сектор рулетки |
|---|-------------|----------|----------------|
| 1 | 723         | 20.4%    | 0–20.4         |
| 2 | 19          | 0.6%     | 20.4–21.0      |
| 3 | 99          | 2.8%     | 21.0–23.8      |
| 4 | 883         | 24.9%    | 23.8–48.7      |
| 5 | 129         | 3.7%     | 48.7–52.4      |
| 6 | 1683        | 47.6%    | 52.4–100       |

## Розыгрыш

Сгенерируем 6 случайных чисел из  $[0, 100]$ : 97, 26, 54, 13, 31, 88.

Выбранные родители:

- 97 → особь 6
- 26 → особь 4
- 54 → особь 6
- 13 → особь 1
- 31 → особь 4
- 88 → особь 6

Родительский пул:  $\{s^6, s^4, s^6, s^1, s^4, s^6\}$ .

## Формирование пар

1.  $(s^1, s^6)$
2.  $(s^4, s^6)$
3.  $(s^6, s^6)$

### 3. Скрещивание (одноточечный кроссинговер)

Пара 1:  $s^1 = 10011$  и  $s^6 = 11101$

Точка разрыва:  $\chi = 4$ .

$s^1 = 100|11$ ,  $s^6 = 111|01$ .

Потомки:

- $s^{1'} = 10001_2 = 17_{10}$ ,  $f(17) = 2 \cdot 17^2 + 1 = 579$
- $s^{2'} = 11111_2 = 31_{10}$ ,  $f(31) = 2 \cdot 31^2 + 1 = 1923$

Пара 2:  $s^4 = 10101$  и  $s^6 = 11101$

Точка разрыва:  $\chi = 2$ .

$s^4 = 10|101$ ,  $s^6 = 11|101$ .

Потомки:

- $s^{3'} = 11101_2 = 29_{10}$ ,  $f(29) = 1683$
- $s^{4'} = 10101_2 = 21_{10}$ ,  $f(21) = 883$

Пара 3:  $s^5 = 11101$  и  $s^6 = 11101$

Точка разрыва:  $\chi = 2$ .

Потомки:

- $s^{5'} = 11101_2 = 29_{10}$ ,  $f(29) = 1683$
- $s^{6'} = 11101_2 = 29_{10}$ ,  $f(29) = 1683$

#### Итог скрещивания

| № | Хромосома | $x$ | $f(x)$ |
|---|-----------|-----|--------|
| 1 | 10001     | 17  | 579    |
| 2 | 11111     | 31  | 1923   |
| 3 | 11101     | 29  | 1683   |
| 4 | 10101     | 21  | 883    |

## 4. Мутация

Вероятность мутации  $p_m = 0.2$ . Для каждого потомка генерируем случайное число из  $[0, 1]$ . Если оно  $< 0.2$ , инвертируем случайно выбранный ген.

| № | Потомок до мутации | Случ. число | Мутация?     | Потомок после мутации | $f(x)$ до | $f(x)$ после |
|---|--------------------|-------------|--------------|-----------------------|-----------|--------------|
| 1 | 10001              | 0.61        | Нет          | 10001                 | 579       | 579          |
| 2 | 11111              | 0.15        | Да (4-й бит) | 11101                 | 1923      | 1683         |
| 3 | 11101              | 0.31        | Нет          | 11101                 | 1683      | 1683         |
| 4 | 10101              | 0.04        | Да (1-й бит) | 00101                 | 883       | 51           |
| 5 | 11101              | 0.71        | Нет          | 11101                 | 1683      | 1683         |
| 6 | 11101              | 0.35        | Нет          | 11101                 | 1683      | 1683         |

## 5. Формирование нового поколения (элитарная стратегия)

Объединяем старую популяцию и потомков, выбираем 6 лучших:

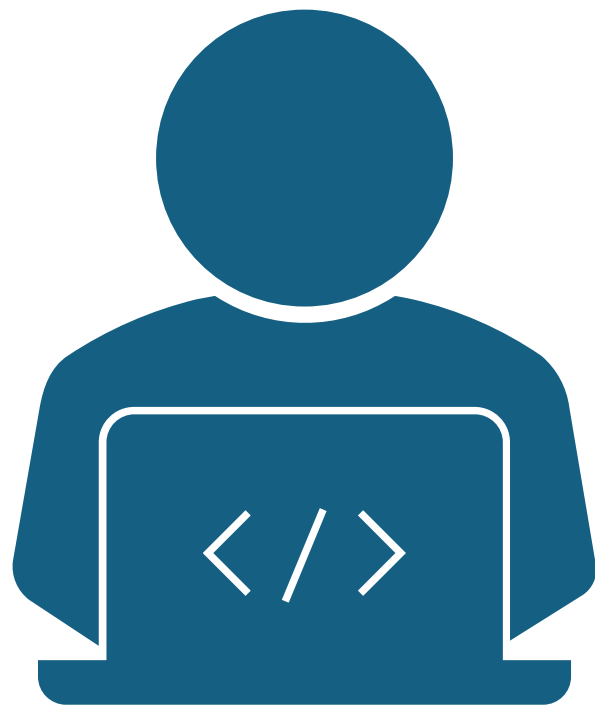
| № | Хромосома | $x$ | $f(x)$ | Источник               |
|---|-----------|-----|--------|------------------------|
| 1 | 11111     | 31  | 1923   | потомок 2 (до мутации) |
| 2 | 11101     | 29  | 1683   | старая особь 6         |
| 3 | 11101     | 29  | 1683   | потомок 3              |
| 4 | 11101     | 29  | 1683   | потомок 5              |
| 5 | 11101     | 29  | 1683   | потомок 6              |
| 6 | 10101     | 21  | 883    | старая особь 4         |

Средняя приспособленность нового поколения:

$$\Phi = \frac{1923 + 1683 \cdot 4 + 883}{6} \approx 1589.7.$$

**Вывод:** за одно поколение алгоритм нашёл глобальный максимум  $x = 31$ ,  $f(31) = 1923$ .

Средняя приспособленность выросла с 589 до 1590.



Демонстрация

# Пример решения на Python



Альтернативы и  
сравнение

## Недостатки генетического алгоритма

1. Нет гарантии нахождения глобального оптимума.
2. Чувствительность к выбору параметров.
  - Слишком большой  $p_m$  → случайный поиск.
  - Слишком малый  $p_m$  → преждевременная сходимость.
  - Малый размер популяции → потеря разнообразия.
  - Большой размер популяции → медленная работа.
1. Проблема преждевременной сходимости.
2. Высокая вычислительная стоимость.
3. Сложность выбора кодирования.
4. Проблема «обманчивых» функций (deception).
5. Проблема «поля для гольфа».

| Метод                     | Когда использовать                                                        | Преимущества                                                                                              | Недостатки                                                                                                 |
|---------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Генетический алгоритм     | Сложные, многомерные, нелинейные задачи; разрывные функции; нет градиента | Не требует производных; устойчив к локальным оптимумам; легко параллелится; работает с любым кодированием | Медленная сходимость; требует подбора параметров; не гарантирует оптимум; высокая вычислительная стоимость |
| Градиентный спуск         | Гладкие, дифференцируемые функции                                         | Быстрая сходимость; простота; много реализаций                                                            | Застревает в локальных минимумах; требует градиента                                                        |
| Имитация отжига           | Большое пространство поиска; многоэкстремальные функции                   | Прост в реализации; может выходить из локальных оптимумов                                                 | Требуется подбора графика температуры; медленный                                                           |
| Метод роя частиц (PSO)    | Непрерывная оптимизация                                                   | Быстрая сходимость; мало параметров                                                                       | Преждевременная сходимость; плохо для дискретных задач                                                     |
| Дифференциальная эволюция | Непрерывная оптимизация                                                   | Простота; эффективность                                                                                   | Не работает для дискретных задач                                                                           |
| Полный перебор            | Малые размерности ( $n < 20$ )                                            | Гарантирует глобальный оптимум                                                                            | Экспоненциальная сложность; неприменим для больших задач                                                   |
| Случайный поиск           | Базовый уровень для сравнения                                             | Простота                                                                                                  | Крайне низкая эффективность                                                                                |

## Когда выбирать генетический алгоритм?

### ГА — хороший выбор, когда

1. Функция недифференцируема или её градиент неизвестен.
2. Функция многоэкстремальна — градиентные методы застрянут в локальном оптимуме.
3. Пространство поиска дискретно (перестановки, бинарные строки).
4. Нужно найти несколько хороших решений, а не одно идеальное.
5. Задача легко распараллеливается — каждую особь можно оценивать независимо.
6. Есть время на вычисления — ГА не быстрый, но надёжный.

### ГА — плохой выбор, когда

1. Функция гладкая и унимодальная — градиентный спуск справится быстрее.
2. Каждая оценка функции очень дорогая — ГА требует много вычислений.
3. Нужна гарантия глобального оптимума — используйте точные методы.
4. Размерность очень высокая (тысячи параметров) — ГА плохо масштабируется.
5. Нет времени на подбор параметров — ГА чувствителен к настройкам.

## Гибридные подходы

### ГА + градиентный спуск

- Идея: ГА ищет хорошую начальную точку, градиентный спуск уточняет.
- Пример: эволюция весов нейросети, затем fine-tuning с помощью backpropagation.

### ГА + K-means

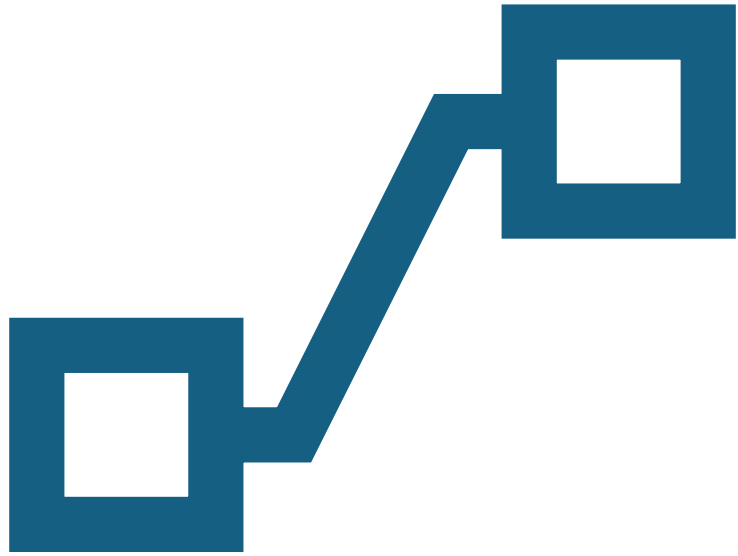
- Идея: ГА подбирает число кластеров  $K$  и начальные центроиды, K-means уточняет.
- Пример: кластеризация с автоматическим выбором  $K$ .

### ГА + дерево решений

- Идея: ГА оптимизирует структуру дерева (глубину, признаки для разбиения).
- Пример: генетическое программирование для синтеза решающих деревьев.

### Меметические алгоритмы

- Идея: комбинация эволюционных операторов с локальным поиском.
- Пример: каждый потомок после мутации подвергается локальной оптимизации.



Заключение

1. **Генетический алгоритм** — мощный эволюционный метод оптимизации, вдохновлённый природой.
2. **Основные компоненты:** популяция, функция приспособленности, селекция, скрещивание, мутация.
3. ГА не гарантирует нахождение глобального оптимума, но часто находит хорошие решения для сложных задач.
4. **Ключевые преимущества:** не требует производных, работает с разрывными и многоэкстремальными функциями, легко параллелится.
5. **Ключевые недостатки:** медленная сходимость, чувствительность к параметрам, нет гарантии оптимума.
6. Реализация возможна «с нуля» на любом языке программирования.
7. В машинном обучении ГА применяется для настройки гиперпараметров, отбора признаков, обучения нейросетей, кластеризации.
8. Гибридные подходы (ГА + градиентный спуск, ГА + K-means) часто дают лучшие результаты, чем каждый метод по отдельности.

**Параллельные ГА:** использование GPU и распределённых вычислений.

**Самоадаптивные ГА:** автоматическая настройка параметров в процессе эволюции.

**Нейроэволюция:** совместная эволюция архитектуры и весов нейросетей.

**Квантовые ГА:** использование квантовых вычислений для ускорения.

## **Основные источники:**

- Holland J.H. Adaptation in Natural and Artificial Systems. – University of Michigan Press, Ann Arbor, 1975. – 183 p.
- Goldberg D.E. Genetic Algorithms in Search, Optimization and Machine Learning. – Addison-Wesley, Reading, MA, 1989. – 412 p.
- Mitchell M. An Introduction to Genetic Algorithms. – MIT Press, Cambridge, MA, 1999. – 158 p.
- Панченко Т.В. Генетические алгоритмы: учебно-методическое пособие / под ред. Ю.Ю. Тарасевича. – Астрахань: Издательский дом «Астраханский университет», 2007. – 87 с.
- Емельянов В.В., Курейчик В.В., Курейчик В.М. Теория и практика эволюционного моделирования. – М.: Физматлит, 2003. – 432 с.
- Гладков Л.А., Курейчик В.В., Курейчик В.М. Генетические алгоритмы: Учебное пособие. – 2-е изд. – М.: Физматлит, 2006. – 320 с.

## **Интернет-ресурсы:**

- Андреев М. Генетический алгоритм. Просто о сложном. – Хабрахабр, 2017.  
URL: <https://habr.com/ru/post/327044/>
- DEAP: Distributed Evolutionary Algorithms in Python. URL: <https://deap.readthedocs.io/>